

GUIA DE DEMOSTRACION EN VIVO

TRADING ROI EXPLAINER & TRANSACTION SIMULATOR API (GEMINI 3.5)

Esta guía contiene todos los pasos ordenados para realizar una demostración en vivo interactiva y exitosa de tu trabajo final de integración. Se enfoca en demostrar el comportamiento de Streaming vs No Streaming, la seguridad de endpoints y la observabilidad con OpenTelemetry.

Paso 1: Demostración de Gemini 3.5 en Consola (Streaming vs No Streaming)

Compara la respuesta síncrona contra la visualización en vivo de tokens.

1. Ejecutar CON STREAMING:

```
python codigo/scripts/demo_gemini_streaming.py --stream
```

- Resalta la velocidad percibida instantánea (<1s).
- Resalta la caja de comprobación de enmascaramiento: 'Maria Gomez' -> 'M**** G****'.

2. Ejecutar SIN STREAMING:

```
python codigo/scripts/demo_gemini_streaming.py
```

- Muestra la espera síncrona estática (20s) para justificar el uso de streaming en producción.

Paso 2: Levantar el Servidor Web e Interfaz Swagger (FastAPI)

Muestra el endpoint cerrado con llave ante accesos no autorizados:

1. Inicia el servidor FastAPI:

```
python -m uvicorn main:app --reload --host 127.0.0.1 --port 8000 --app-dir "codigo"
```

2. Abre la documentación interactiva en el navegador:

```
http://127.0.0.1:8000/docs
```

3. Demuestra la protección X-API-Key: Sin cabecera el servidor bloquea con error HTTP 401. Al ingresar la clave 'trading-secret-key-123' en la autorización superior, retorna 200 OK.

Paso 3: Demostración de Observabilidad con OpenTelemetry (OTel)

Demuestra el sistema de diagnóstico empresarial de trazas y logs en tiempo real:

1. Desglose de Spans: Muestra los logs en consola '[OTel Span]' con las latencias parciales de cada capa (conector, adaptador, Gemini).
2. Correlación de Logs: Resalta que cada registro de log posee el [TraceId: ... | SpanId: ...].
3. Diagnóstico de Errores: Envía una solicitud con un ticker no válido como 'XYZ' y observa como OTel marca el Span en 'ERROR' registrando la excepción sin tumbar el servidor.
4. Simulación de Falla en Base de Datos: Cambia 'mock_db.json' a 'Solo Lectura' en Windows. Verás cómo el sistema aborta de inmediato la llamada a Gemini para resguardar la consistencia financiera y le indica al cliente que vuelva a intentar (Retornando error 503).

Paso 4: Pruebas y Medición de Latencias (p50 y p95)

1. Ejecutar las pruebas unitarias y de enmascaramiento de datos:

```
python -m pytest codigo/tests/test_trading_api.py -v
```

2. Ejecutar el benchmark para medir p50 y p95 latencia (10 llamadas consecutivas):

```
python codigo/scripts/benchmark_latencia.py
```

Se generará la tabla de percentiles en pantalla y se guardará en benchmark_results.json.